Research Square

Preprints are preliminary reports that have not undergone peer review.
They should not be considered conclusive, used to inform clinical practice,
or referenced by the media as validated information.

A Hybrid RAG System Integrating Knowledge Graph and Vector Retrieval: Based on Solution Technical Documents

Cheonsu Jeong

paripal@korea.ac.kr

SAMSUNG SDS

Research Article

Keywords: Hybrid RAG, Knowledge Graph, Vector Retrieval, Information Retrieval, Reciprocal Rank Fusion (RRF)

Posted Date: April 22nd, 2026

DOI: https://doi.org/10.21203/rs.3.rs-9476748/v1

License: This work is licensed under a Creative Commons Attribution 4.0 International License. Read Full License

Additional Declarations: The authors declare no competing interests.

A Hybrid RAG System Integrating Knowledge Graph and Vector Retrieval: Based on Solution Technical Documents

Cheonsu Jeong[1,*]

[1]*AX Center, SAMSUNG SDS | csu.jeong@samsung.com | Seoul 05510, South Korea*

**Corresponding Author: Dr. Cheonsu Jeong | paripal@korea.ac.kr*

27 March 2026

Abstract

As enterprise environments increasingly manage large volumes of technical documentation across multiple heterogeneous sources, users face growing challenges in efficiently locating relevant information. This study proposes and implements a hybrid Retrieval-Augmented Generation (RAG) system that integrates a knowledge graph with vector retrieval to enhance information access in solution technical documents. The proposed system combines a Neo4j-based knowledge graph with a ChromaDB-based vector database, enabling both structural relationship exploration and semantic similarity search. A domain-specific knowledge graph is constructed by extracting entities and relationships from technical documents, including cross-document references derived from HTML anchor links. In parallel, vector embeddings are generated using a multilingual embedding model to support semantic retrieval across diverse document types. The system employs a hybrid ranking mechanism based on Reciprocal Rank Fusion (RRF) to effectively integrate graph-based and vector-based retrieval results. A seven-stage query processing pipeline is designed to analyze user queries, perform graph traversal and vector search, fuse results, and generate natural language responses using a large language model. Experimental evaluation on a multi-source technical document dataset demonstrates that the proposed hybrid approach significantly improves retrieval performance compared to single-method baselines, particularly in complex queries requiring cross-document reasoning. The results indicate that integrating knowledge graph and vector retrieval within a hybrid RAG framework provides an effective solution for enhancing information retrieval in enterprise technical documentation environments.

Keywords: *Hybrid RAG, Knowledge Graph, Vector Retrieval, Information Retrieval, Reciprocal Rank Fusion (RRF)*

1. Introduction

With the acceleration of digital transformation, the volume of technical documentation produced and managed by enterprises has grown substantially. Users increasingly encounter difficulties in accurately and efficiently locating the information they require. In particular, identifying complex relationships and meanings embedded within unstructured textual data presents a considerable challenge. Enterprise software solutions—such as Robotic Process Automation (RPA) platforms—are typically accompanied by diverse forms of technical documentation, including user manuals, administration manuals, installation guides, troubleshooting guides, and training materials. When such solution technical documents are managed independently and in a distributed manner, users must manually navigate multiple documents to retrieve the required information, thereby reducing operational efficiency and delaying error resolution (Petroni et al., 2021). Traditional keyword-based retrieval systems rely on lexical matching within documents and are therefore inherently limited in their ability to perform automatic cross-referencing between semantically related documents. For example, a user searching for a 'scheduler error configuration scenario' would need to simultaneously consult error code information from the troubleshooting guide, scheduler function descriptions from the user manual, environment configuration details from the installation guide, and practical scenarios from the training materials. Conventional information retrieval systems, which provide results based on keyword matching or statistical similarity, frequently fail to deliver satisfactory responses to such in-depth queries. This limitation stems from an insufficient capacity for contextual understanding and inferential reasoning over information.

Recent advances in large language models (LLMs) have brought considerable attention to Retrieval-Augmented Generation (RAG) technology. RAG mitigates the hallucination problem inherent in LLMs and enables the incorporation of up-to-date domain knowledge by retrieving relevant documents from external knowledge stores for use in response generation (Lewis et al., 2020; Gao et al., 2023). However, RAG systems based solely on vector similarity search exhibit limitations in exploring structural relationships and semantic pathways across documents. To address this, Graph RAG—which combines knowledge graphs with RAG—has been proposed (Edge et al., 2024; Pan et al., 2024). In particular, the use of LangGraph in agent-based Advanced RAG systems incorporating graph technology has demonstrated improved retrieval accuracy (Jeong, 2024).

This study designs, implements, and evaluates a hybrid Graph RAG system targeting multi-type solution technical documents (HTML, PDF), combining a Neo4j knowledge graph database with a ChromaDB vector database. The core technical contributions include: (1) automated graph transformation of cross-reference relationships derived from HTML anchor links, (2) RRF-based hybrid retrieval ranking, and (3) a seven-stage query processing pipeline.

The objectives of this research are threefold: first, to present the implementation of a hybrid knowledge retrieval system based on Graph RAG architecture within an enterprise technical documentation environment; second, to implement an integrated architecture combining Neo4j graph database and ChromaDB vector database with an RRF-based fusion retrieval algorithm; and third, to empirically demonstrate the superiority of hybrid retrieval through performance evaluation on real solution technical documents.

The scope of this study encompasses technical documents including online manuals (user, administration, installation, and troubleshooting guides) and training materials (introductory, advanced, and manual-based courses) for a software solution.

The remainder of this paper is organized as follows. Chapter 2 reviews related work on RAG, Graph RAG, agent-based Advanced RAG, knowledge graph construction, and hybrid retrieval. Chapter 3 presents the system design and architecture. Chapter 4 describes implementation details and system interface screenshots. Chapter 6 presents conclusions and directions for future research.

2. Related Work

This section reviews the theoretical foundations and recent research trends relevant to the core components of hybrid RAG: knowledge graphs, vector retrieval, and the RRF algorithm.

2.1 Advances in Retrieval-Augmented Generation (RAG)

RAG was first proposed by Lewis et al. (2020) as an open-domain question answering framework that combines retrieval and generation, enabling LLMs to leverage external knowledge beyond their training period. The original RAG architecture integrated a Dense Passage Retriever (DPR) with a sequence-to-sequence generative model. Subsequently, Gao et al. (2023) systematically categorized RAG development into three stages: Naive RAG, Advanced RAG, and Modular RAG.

Naive RAG relies on simple chunking and vector similarity search, exhibiting limitations in both precision and recall. Because it utilizes pre-loaded knowledge without reprocessing, it may produce contextual misinterpretations and biased outputs, and is also unable to reflect real-time data following RAG configuration (Jeong, 2024). Advanced RAG addresses these shortcomings through pre-retrieval and post-retrieval optimization, while Modular RAG adopts an architecture in which retrieval and generation modules can be flexibly combined—a direction consistent with the hybrid approach of this study (Gao et al., 2023).

Research on RAG in Korean-language contexts has also been increasing. Studies range from introducing procedures for building RAG systems utilizing internal corporate documents with vector-based approaches (Jeong, 2023), to constructing RAG-based question answering systems for Korean legal documents and analyzing the effects of retrieval strategies and chunking approaches on response quality (Kim & Lee, 2023), to demonstrating the effectiveness of the multilingual embedding model BGE-M3 applied to Korean technical documents—confirming high retrieval performance in Korean domains that incorporate English technical terminology (Park et al., 2024).

2.2 Agent-Based Advanced RAG

Agent-based approaches have attracted significant attention as a means of overcoming the limitations of conventional RAG. Jeong (2024) proposed a methodology for implementing an agent-based Advanced RAG system utilizing graph technology. In that work, LangGraph was employed to evaluate the reliability of retrieved information and to integrate diverse information sources, thereby generating more accurate and enhanced responses. Notably, the approach of combining an agent that evaluates response content with additional web searches for accuracy improvement provided practical guidance for enterprise service implementation. The research proposed a four-stage methodology comprising: (1) defining RAG system requirements, (2) designing and visualizing graph structures (nodes and edges), (3) collecting and refining textual data from multiple sources, and (4) implementing a LangGraph-based agent pipeline.

The present study extends this methodology by developing a hybrid architecture that integrates a Neo4j graph database with a ChromaDB vector database. The core of agent-based RAG lies in implementing a system that transcends simple retrieval-generation pipelines, enabling dynamic decision-making—including reliability evaluation of retrieved results, re-retrieval when necessary, and multi-source information integration.

2.3 Knowledge Graph-Based Retrieval

A knowledge graph (KG) is a data model that represents real-world knowledge as a structured network of entities (nodes) and relationships (edges). Representative examples include Google's Knowledge Graph (Singhal, 2012) and Wikidata (Vrandečić & Krötzsch, 2014). Neo4j is widely used as a graph database, enabling multi-hop relationship traversal through the Cypher query language (Robinson et al., 2015).

Diverse research has been conducted on the application of knowledge graphs in the field of enterprise document management. Xu et al. (2022) converted procedural information from technical manuals into a graph structure to implement a step-by-step task guidance system, demonstrating meaningful results in terms of procedural completeness compared with simple retrieval. Zhang et al. (2023) proposed a knowledge graph-based fault diagnosis system for software documentation, experimentally verifying that relationship modeling between error codes and resolution procedures improves diagnostic accuracy.

Research on KG-RAG in manufacturing domains has also been active. Amugongo et al. (2024) identified three major challenges for KG-RAG models: insufficient integration between structured and unstructured knowledge, limitations in multi-hop reasoning path selection accuracy, and the absence of standardized evaluation protocols. From an ontology design perspective, domain-specific ontology construction methodologies extending Gruber's (1993) definition have also been investigated (Noy & McGuinness, 2001).

2.4 Recent Trends in Graph RAG Research

Research leveraging knowledge graphs to overcome the limitations of existing RAG systems and enhance LLM reasoning capabilities has been actively conducted. Knowledge graphs provide structured knowledge by representing entities and their interrelationships as nodes and edges (Hogan et al., 2021). This structured knowledge assists LLMs in understanding complex interconnections among information and performing more accurate and in-depth reasoning.

Edge et al. (2024) of Microsoft Research proposed the Graph RAG framework, reporting that community summary-based global search utilizing LLM-generated knowledge graphs outperforms conventional RAG on complex reasoning queries. In that study, Graph RAG improved the comprehensiveness metric for topic summarization queries spanning multiple documents by 72%. Graph RAG retrieves query-relevant information from the knowledge graph and supplies it to the LLM for response generation. This approach transcends simple text retrieval, improving answer quality for multi-step reasoning and complex queries by leveraging the structural properties of knowledge graphs.

Peng et al. (2024) formalized the GraphRAG workflow into three stages in a comprehensive survey: Graph-based Indexing (G-Indexing), Graph-guided Retrieval (G-Retrieval), and Graph-enhanced Generation (G-Generation), and demonstrated that agent-based approaches can improve quality at each stage.

Pan et al. (2024) categorized three paradigms for integrating knowledge graphs with large language models (LLMs): KG-augmented LLMs, LLM training leveraging KGs, and KG construction using LLMs. Zhang et al. (2024) conducted a comparative analysis of state-of-the-art indexing methods across three multihop reasoning datasets (MuSiQue, 2WikiMultiHopQA, and HotpotQA), demonstrating that the proposed SiReRAG consistently outperformed existing approaches with an average F1 score improvement of 1.9%. Notably, SiReRAG achieved approximately 5% higher average F1 scores than RAPTOR and improved F1 scores by up to 8.3% over HippoRAG on the MuSiQue dataset.

Beyond indexing, reranking constitutes another critical component of the RAG pipeline (Zhang et al., 2024). Ong et al. (2025) proposed an intelligent reranking framework that enhances retrieval quality in RAG systems by effectively incorporating follow-up query predictions to better capture user intent, thereby optimizing the overall RAG process. Furthermore, many techniques employed in RAG systems exhibit language-dependent characteristics, and their effectiveness may vary across different linguistic contexts (Elkiran & Rasheed, 2026).

In the educational domain, Ma et al. (2025) proposed KA-RAG, a framework that integrates structured knowledge graphs with an agentic RAG workflow to implement a process-oriented question-answering system. By adopting a dual retrieval strategy that combines symbolic graph reasoning with dense semantic search, KA-RAG achieved a retrieval accuracy of 91.4%. Collectively, these advances reflect a broader trend in which Graph RAG systems are evolving toward hybrid architectures that integrate graph databases with vector databases to achieve further improvements in retrieval accuracy (Jeong, 2026).

2.5 Hybrid Retrieval Ranking: Reciprocal Rank Fusion (RRF)

Hybrid retrieval combines lexical-based search (e.g., BM25) with dense vector retrieval, allowing the two approaches to complement each other's limitations (Bruch et al., 2023). Reciprocal Rank Fusion (RRF), proposed by Cormack et al. (2009), is a method that integrates multiple ranked lists by summing the reciprocal ranks of each document. RRF with k=60 guarantees a theoretical lower bound and has the characteristic of minimizing rank bias. RRF is a powerful technique that can compute integrated scores based on rank information regardless of the scoring scheme of each retrieval source. The integrated score is formulated as follows:

$$Score(d) = \sum_{r \in R} \frac{1}{k + rank_r(d) + 1}$$

where *rank_r(d)* denotes the rank of document *d* in list r (starting from 0), and *k* is a smoothing constant that controls the rate of score decay. The retrieval behavior of the system varies with the setting of *k* as follows:

- Small *k* values (e.g., 10–20): Assign very high weights to top-ranked results. Results that reach the top positions in a particular retrieval method exert a dominant influence on the final output.
- Large *k* values (e.g., 60–100): The score difference between ranks is attenuated. Rather than results dominated by one retrieval method, 'consensus results' that rank highly across both retrieval methods receive higher scores, enabling more stable integration.

For example, when *k=10*, the score ratio between the 1st and 10th positions is approximately 1.8, indicating a tendency to concentrate on top results. In contrast, when *k=60*, this ratio narrows to 1.06, allowing lower-ranked results to contribute more equitably to the final output. In this study, *k=60*—the generally recommended standard value—was adopted as the default setting to maximize the complementarity of the two retrieval models. This configuration ensures that the precision-oriented relationship retrieval results from the knowledge graph and the semantic similarity retrieval results from the vector database are balanced and integrated without undue bias toward either source. The system also enables users to manually adjust the k parameter according to knowledge type.

In this study, RRF is applied to integrate Neo4j graph database traversal results with ChromaDB vector retrieval results. This represents a novel hybrid paradigm extending from the conventional BM25 and dense retrieval combination to graph traversal, differentiating itself by simultaneously exploiting structural relationship information and semantic similarity. The dense embedding-based retrieval research originating from Karpukhin et al.'s (2020) DPR has advanced to multilingual models such as BAAI/BGE-M3 (Chen et al., 2024), providing a theoretical basis for optimal embedding selection in Korean-English mixed technical domains.

2.6 Differentiation from Existing Research

Existing Graph RAG research has primarily targeted monolingual text corpora or general-domain knowledge graphs based on Wikipedia. While prior research proposed a methodology for enterprise application of LangGraph-based agent RAG, the present study builds upon that foundation to introduce the following differential contributions:

- Conversion of anchor links from HTML Single Page Application (SPA) structures into REFERENCES relationships in Neo4j.
- Design and application of a 9-node/11-edge ontology for solution technical documents.

- Implementation of a multi-source pipeline that integrates HTML and PDF heterogeneous sources into a unified knowledge graph and vector index.

These contributions are complementary to existing prior work while presenting a new architectural reference model from an enterprise practical application perspective.

3. Research Methodology

This section presents the design of a hybrid RAG system that combines structured data from a knowledge graph with unstructured numerical data from a vector database. The overall architecture, data modeling, retrieval ranking, and constituent components of the response generation pipeline are described step by step for efficient navigation of technical documentation.

This chapter presents the design of a hybrid RAG retrieval system that integrates structural data from a knowledge graph with semantic vector representations from a vector database, as illustrated in Figure 1. The system is designed to maintain the currency of technical documentation, and the entire process is carried out through a five-layer architecture and a seven-stage query processing pipeline.

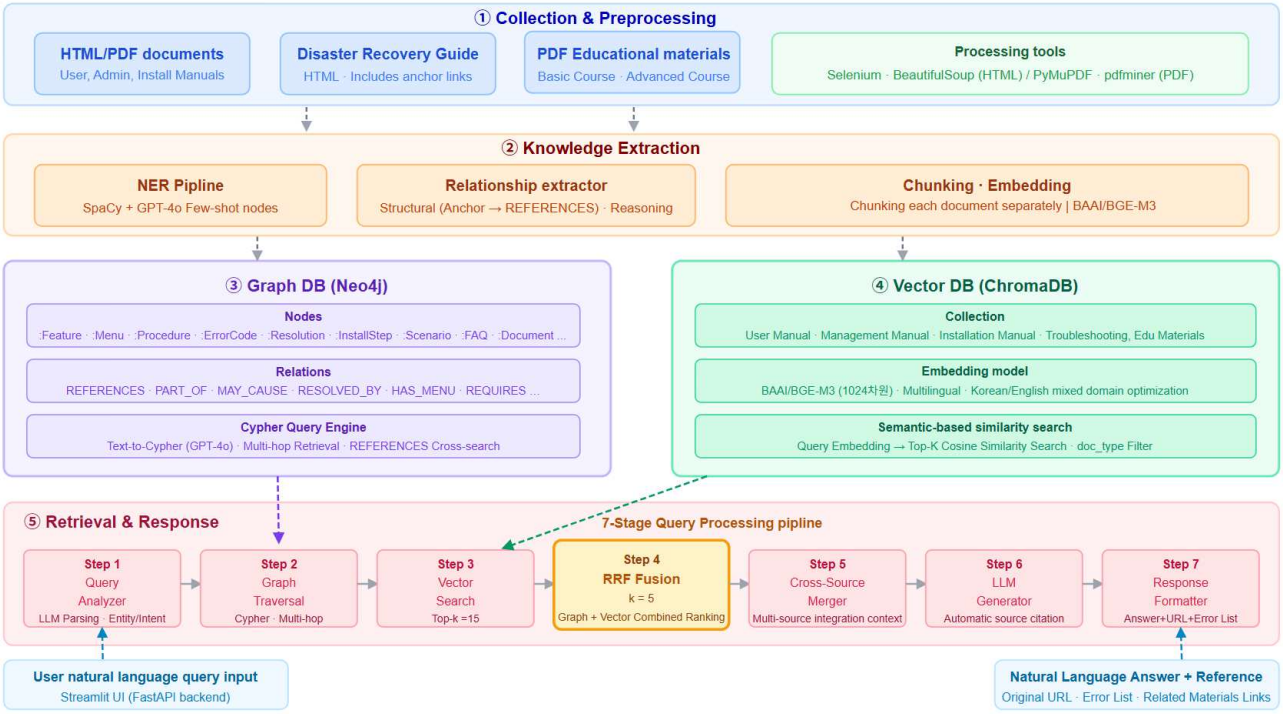

Figure 1: Hybrid Retrieval Architecture Based on Graph RAG

The following sections describe in detail the overall proposed architecture, data modeling, retrieval ranking mechanism, and the constituent components of the answer generation pipeline for efficient technical document navigation.

3.1 Overall System Architecture

The hybrid Graph RAG system proposed in this study is designed as a five-layer architecture: (1) Collection and Preprocessing Layer, (2) Knowledge Extraction Layer, (3) Graph DB Layer, (4) Vector DB Layer, and (5) Retrieval and Response Generation Layer. Each layer operates as an independent service.

Table 1. Five-Layer System Architecture Configuration

Layer	Component Modules	Core Technologies	Primary Outputs
① Collection & Preprocessing	HTML Crawler, PDF Parser, Chunking & Metadata Tagger	Selenium, BeautifulSoup, PyMuPDF	Structured JSON chunks, Anchor link map
② Knowledge Extraction	NER Pipeline, Relation Extractor	SpaCy + GPT-4o Few-shot	Entity list, Relation triples
③ Graph DB	Node/Edge Loader, Cypher Traversal Engine	Neo4j 5.x, APOC, Py2neo	Integrated knowledge graph, Cypher API
④ Vector DB	Embedding Generator, ChromaDB Indexer	ChromaDB, BAAI/BGE-M3	7 collections, Top-K retrieval
⑤ Retrieval & Response	Hybrid Retriever, RRF Engine, LLM Generator	LangChain, GPT-4o, FastAPI, Streamlit	Integrated ranked results, Natural language responses

The service connectivity of the system is designed such that user browsers access the system through the Streamlit UI (port 8501), which connects directly to Neo4j (port 7687) and ChromaDB (port 8000), or performs integrated retrieval through the FastAPI backend (port 8080). All services are designed to operate in on-premises secure environments.

3.2 Layer-by-Layer Architecture Design

3.2.1 Data Collection and Preprocessing Module

This module is responsible for collecting various forms of unstructured documents (e.g., PDFs, web pages) and preprocessing them into formats suitable for LLM use and knowledge graph construction. Its primary functions include:

- Document Crawling: Collects data from online sources such as web pages. This includes the capability to extract text content from web pages based on specific URLs.
- PDF Parsing: Extracts text, images, tables, and other information from PDF files. In particular, textual data is converted into structured form for subsequent entity and relationship extraction.
- Text Preprocessing: Applies preprocessing steps including stopword removal, morphological analysis, and Named Entity Recognition (NER) to extracted text data. This process is essential for improving the accuracy of knowledge graph construction.

3.2.2 Knowledge Extraction

The knowledge extraction stage is a core process for generating the foundational data of the knowledge graph by identifying meaningful entities and structural relationships from collected unstructured technical document data. This study designed the following NER pipeline and relation extraction process for precise

knowledge base construction.

(1) NER (Named Entity Recognition) Pipeline

The NER pipeline classifies key terms within technical documents into predefined entity types. The system is designed to identify 9 node types reflecting the characteristics of solution[1] technical documents, including 'Component,' 'Function,' 'Error_Code,' and 'Parameter.' The pipeline employs the following strategies:

- Hybrid Recognition Strategy: Combines deep learning-based recognition using language models with domain-specific dictionary-based recognition. This ensures that proprietary nouns such as 'Brity' and 'Designer,' as well as complex technical terms, are captured without omission.
- Contextual Chunking: Entities are extracted while preserving semantic cohesion rather than performing simple text segmentation. This approach maintains contextual information about which parent module a specific function belongs to during named entity identification.

(2) Relation Extractor

The relation extractor constructs the edges of the knowledge graph by generating logical connections between identified entities. To reflect the hierarchical structure of technical documents and inter-document connectivity, two approaches were applied:

- Structural Extraction: Analyzes HTML tag hierarchies (DOM tree) and anchor links (<a> tags) within documents. When a link to another guide page is found within a document, it is automatically converted into a REFERENCES or RELATED_TO relationship edge in the knowledge graph. This is a core contribution that makes cross-document references visible.
- Linguistic Inference: Defines dependency relationships between entities by analyzing verbs and predicates within sentences. USES_PARAMETER relationships are derived from sentences such as 'Function A uses Parameter B,' and PART_OF relationships are extracted from expressions such as 'A is a submenu of B,' thereby completing the hierarchical structure of knowledge.

3.2.3 Knowledge Graph Schema Design

This system utilizes Neo4j as its graph database to store and navigate complex associative relationships between extracted knowledge objects. The knowledge graph schema was designed based on domain analysis of solution technical documents to align with Neo4j's data model.

(1) Node/Edge Loader

The node/edge loader performs bulk loading of intermediate data in JSON or CSV format produced by the preceding knowledge extraction pipeline into the graph database schema.

- Schema-based Mapping: Data is classified according to the 9 predefined node types (Component, Function, Parameter, etc.) and 11 relationship edge types (REFERENCES, PART_OF, USES, etc.). To prevent duplicate entity creation, MERGE operations based on unique identifiers are performed to maintain data integrity.
- Property Optimization: In addition to the entity name, each node stores metadata such as raw text fragments, source URLs, and document types as properties. This assists the LLM in accurately citing the basis for its responses during the generation stage.
- Cross-Reference Graphification: Reference relationships extracted from HTML anchor links are connected as REFERENCES edges, forming a knowledge mesh across distributed documents.

(2) Cypher Traversal Engine

The Cypher traversal engine is a core module that converts user natural language queries into Cypher—the

[1] Brity Automation: Samsung SDS's Hyperautomation Platform

graph-specific query language—or executes structural traversals for hybrid retrieval.

- Text-to-Cypher Conversion: LLM (GPT-4o) is utilized to analyze user query intent and dynamically generate Cypher queries conforming to the knowledge graph schema. For example, a question such as 'What is the configuration pipeline for Function A?' is converted into a path traversal query of the form (n:Function {name:'A'})-[:HAS_STEP]->(m:Step).
- Multi-hop Retrieval: Tracks 'relationships of relationships' that cannot be found through simple keyword search. Resolution documents linked to a specific error code, and the parent guidelines referenced by those documents, are retrieved in a single query to secure rich context.
- Hybrid Interface: Extracted graph path data is returned alongside vector DB retrieval results to serve as the foundational material for final ranking by the RRF engine.
- The node types consist of 9 types (:Feature, :Menu, :Procedure, :ErrorCode, :Resolution, :InstallStep, :Scenario, :FAQ, :Document), and the relationship types consist of 11 types (HAS_MENU, REQUIRES_STEP, MAY_CAUSE, RESOLVED_BY, DEMONSTRATED_IN, ANSWERED_BY, REFERENCES, REQUIRES, RELATES_TO, CONFIGURED_BY, PART_OF).

Table 2. Neo4j Node Types and Key Properties

Node Label	Example Instances	Key Properties
:Feature (Function)	Scheduler configuration, Bot deployment	name, category, doc_source, anchor_id, url
:Menu (Menu path)	Settings > Automation > Scheduler	path, depth, parent_id, doc_source
:Procedure (Procedure)	5-step installation procedure	step_order, action, precondition, doc_source
:ErrorCode (Error)	ERR-4021: Connection timeout	code, message, severity, category
:Resolution (Resolution)	Allow firewall port 8080	steps, verification, related_procedure
:InstallStep (Install step)	DB install → WAS install	step_no, prerequisite, command
:Scenario (Scenario)	Basic bot creation exercise	title, level, objective, doc_source
:FAQ	Q: How to check scheduler not running?	question, answer, related_feature
:Document (Document)	User manual	type, version, source_type, base_url

The REFERENCES relationship is central to the schema design. HTML internal anchor links (<a href='#anchor_id'>) are collected during the crawling phase and converted into REFERENCES edges in Neo4j, enabling cross-referencing between manuals to be traversed as graph paths. For example, when the 'Bot Execution Settings' section of the user manual references the 'Bot Execution Error' section of the troubleshooting guide, this link is automatically converted into a REFERENCES edge.

3.2.4 Vector Collection and Chunking Strategy

The vector database uses ChromaDB and adopts a 7-collection structure consisting of 6 source-type-specific independent collections and 1 unified integrated collection. The BAAI/BGE-M3 (1024-dimensional) model was selected as the embedding model. Chunking strategies were differentiated by source type. The

troubleshooting guide adopts error-code-unit independent chunks (256 tokens/overlap 32), while function-unit (512/64) and procedure-unit (384/48) chunking are predominantly applied to other document types. Training materials are chunked by scenario unit (768/96) to prevent disruption of instructional flow.

3.2.5 Retrieval and Response Module Design: Hybrid Retrieval with RRF

The hybrid retrieval integrates Neo4j Cypher traversal results with ChromaDB vector retrieval results using the RRF algorithm. The RRF score is computed using the following formula:

$$RRFscore(d) = \sum_{r \in R} \frac{1}{k + r(d)}, k = 60$$

where *d* is the document, *R* is the set of rankers, *r(d)* is the rank of document *d* in ranker r, and *k* is a constant with *k=60* as the standard hyperparameter. Graph traversal results and vector retrieval results are each processed as separate ranked lists to compute the RRF integrated score.

3.2.6 User Interface Design

A web-based interface is provided that enables users to interact with the system. This module includes the following features:

- Query Input: Provides an interface through which users can submit questions in natural language form.
- Response Output: Presents LLM-generated responses to users in a clear and readable format.
- Graph Visualization: Visually represents portions of the constructed knowledge graph or sections relevant to the query, enabling users to intuitively understand relationships among information.
- System Configuration and Monitoring: Provides an administrator interface for configuring and monitoring various system functions, including Graph DB configuration, crawling settings, and PDF parsing result verification.

3.2.7 Seven-Stage Query Processing Pipeline

User query processing is implemented as a 7-stage pipeline as shown in Table 3. As confirmed in the processing stage visualization in Figure 3, the progress of each stage is displayed in real time in the UI.

Table 3. Seven-Stage Query Processing Pipeline

Stage	Module	Processing Description
Step 1	Query Analyzer	LLM parsing → Entity: {install, admin account} / Intent: procedure + error exploration
Step 2	Graph Traversal	Cypher: InstallStep→Procedure→Feature(AdminConfig)→ErrorCode (3-hop)
Step 3	Vector Search	Query embedding → ba_unified Top-15 + doc_type filter applied
Step 4	RRF Fusion	Graph traversal results + Vector results integrated ranking via RRF (k=60)
Step 5	Cross-Source Merger	Integrated context construction from installation manual + admin manual + troubleshooting guide + training materials

Step 6	LLM Generator	GPT-4o response generation based on integrated context + automatic source citation
Step 7	Response Formatter	Response + manual URL + error list + exercise links returned

4. System Implementation

This section provides a detailed description of the actual implementation of the proposed Graph RAG system and its key interface screens. The system was developed using Python, and the user interface was constructed using Streamlit. Neo4j Graph Database was employed for knowledge graph storage, ChromaDB was used as the vector database, and the OpenAI GPT model was integrated as the LLM.

4.1 Development Environment and Technology Stack

- Programming Language: Python 3.9+
- Web Framework: Streamlit
- Databases: Neo4j, ChromaDB
- LLM Integration: OpenAI API
- PDF Parsing: PyPDF2, pdfminer.six
- Web Crawling: BeautifulSoup, Requests, Selenium
- Vector Embeddings: Sentence-Transformers, BAAI/BGE-M3

4.2 System Initialization and Graph DB Integration

To use the system, an initial configuration connecting to Graph DB (Neo4j) is required. Users can configure the database connection by entering the connection information (URI, username, password) for Neo4j AuraDB or a local Neo4j instance. This configuration is an essential step for accessing the system's core knowledge store. Figure 2 shows the screen for configuring and connecting to a Graph DB instance in Neo4j Desktop. Upon initial system launch, Neo4j Desktop is executed and either an existing database is imported or a new instance is created. Setting the connection URI (neo4j://127.0.0.1:7687) and authentication credentials enables the Streamlit UI to automatically connect to Neo4j.

Figure 2: Graph DB Configuration Screen

4.3 Initial Screen and System Startup

The initial execution screen displayed in the terminal upon system startup is shown in Figure 3. When the Streamlit application starts, the initialization status of each service is displayed sequentially: Neo4j connection success (graph.graph_builder: Neo4j connection successful), ChromaDB connection success (vectordb.vector_store: ChromaDB connection successful), and BAAI/BGE-M3 embedding model load (391/391). After successful startup, the UI is accessed at http://192.168.0.21:8501.

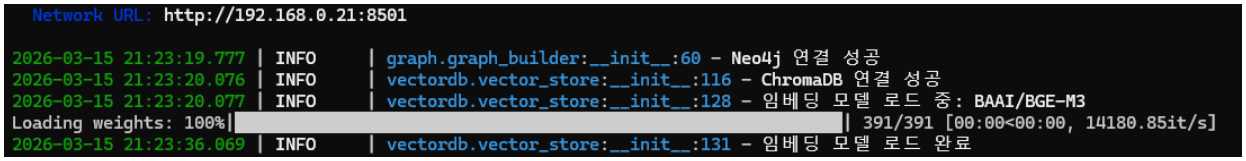

Figure 3: System Initial terminal Screen

4.4 Main Retrieval and Question-Answer Interface

The main retrieval tab screen of the UI, shown in Figure 4, presents the Graph RAG hybrid knowledge retrieval interface. At the top, example query buttons are provided—'How do I configure the scheduler?', 'How to resolve error code 4021?', 'Bot deployment procedure and related exercises,' 'Initial administrator configuration after installation,' and 'Step-by-step workflow creation guide'—allowing users to initiate searches quickly. The left sidebar displays the list of knowledge sources (user manual, administration manual, installation manual, troubleshooting guide, basic course training materials, E2E advanced training materials) and retrieval settings (RRF k value, number of results).

For the query 'How do I configure the scheduler?', the retrieval results returned were: response time 3.83 seconds, 1 graph result, 15 vector results, and confidence 100%. The processing stages (Query Analysis → Neo4j Graph Traversal → ChromaDB Vector Search → RRF Fusion → Integrated Context Construction → LLM Response Generation) are visualized in the right panel. The response area outputs a 6-step procedure: accessing the administration menu, selecting Scheduler, registering a new schedule, selecting the execution bot, configuring the execution cycle, and saving. The referenced sources section at the bottom automatically cites URL links from the user manual, installation prerequisites, and basic course training materials.

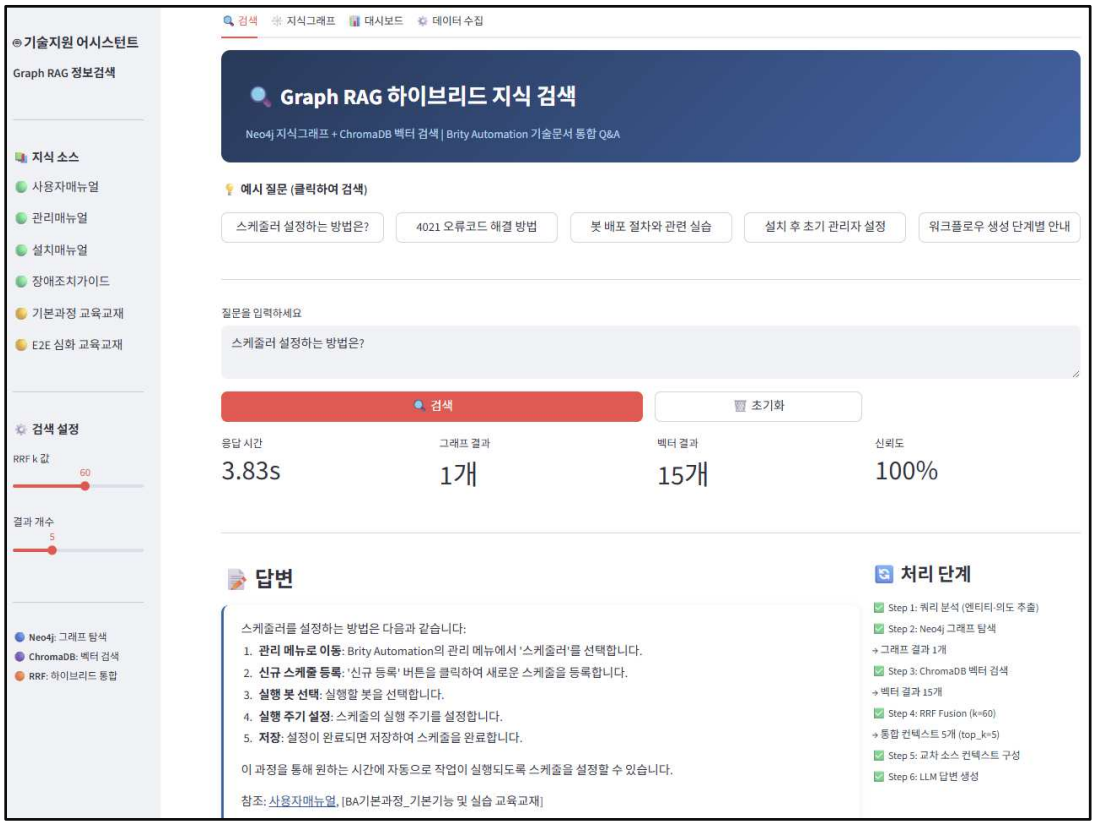

Figure 4: Main Retrieval Screen and Question-Answer Results

The RRF Score contributes to improving the quality of final responses by integrating rankings from diverse retrieval results and evaluating the relative importance of each result. The system presents RRF Scores visually, enabling users to understand which information items played a major role in the LLM's response generation. In the vector results tab of Figure 5, the source document name and similarity score (0–1) for each chunk are displayed as bar graphs. In the example, source-specific relevance scores such as [User Manual] score: 0.724 and [BA Basic Course Training Materials] score: 0.720 are visualized.

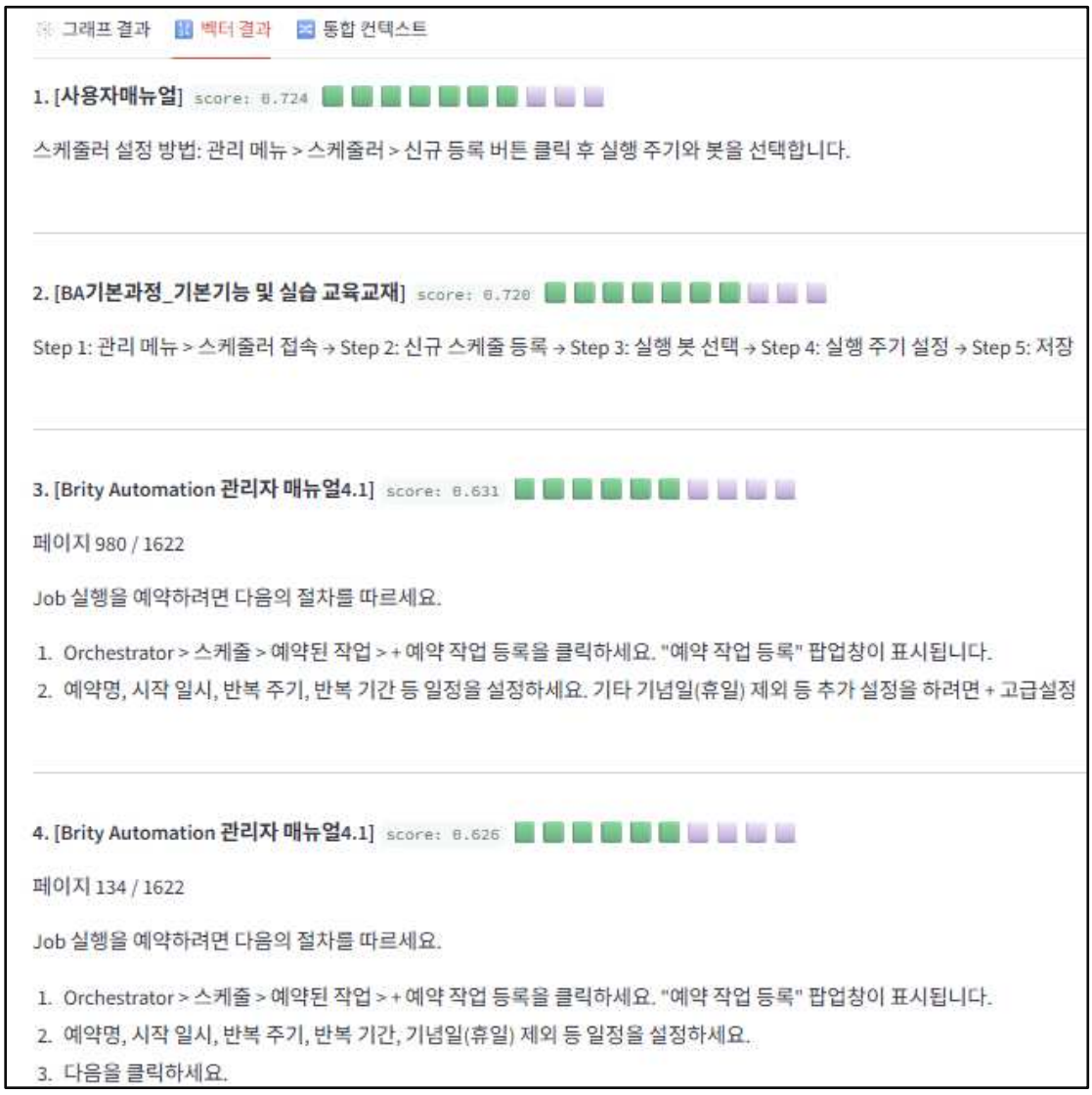

Figure54: Vector Results Interface

The RRF Score interface in Figure 6 displays the RRF Score for each retrieval result along with the source of the information. Figure 6 shows the vector retrieval results and RRF integrated ranking interface. In the integrated context tab, both RRF Score and source type (graph/vector) are displayed together. The top-ranked result is a graph result (Feature: Scheduler) with RRF Score: 0.0164, followed by vector results (user manual, basic course training materials, administration manual) integrated in order of RRF score. This interface enables users to intuitively grasp the relative contributions of graph traversal and vector retrieval in the hybrid retrieval process.

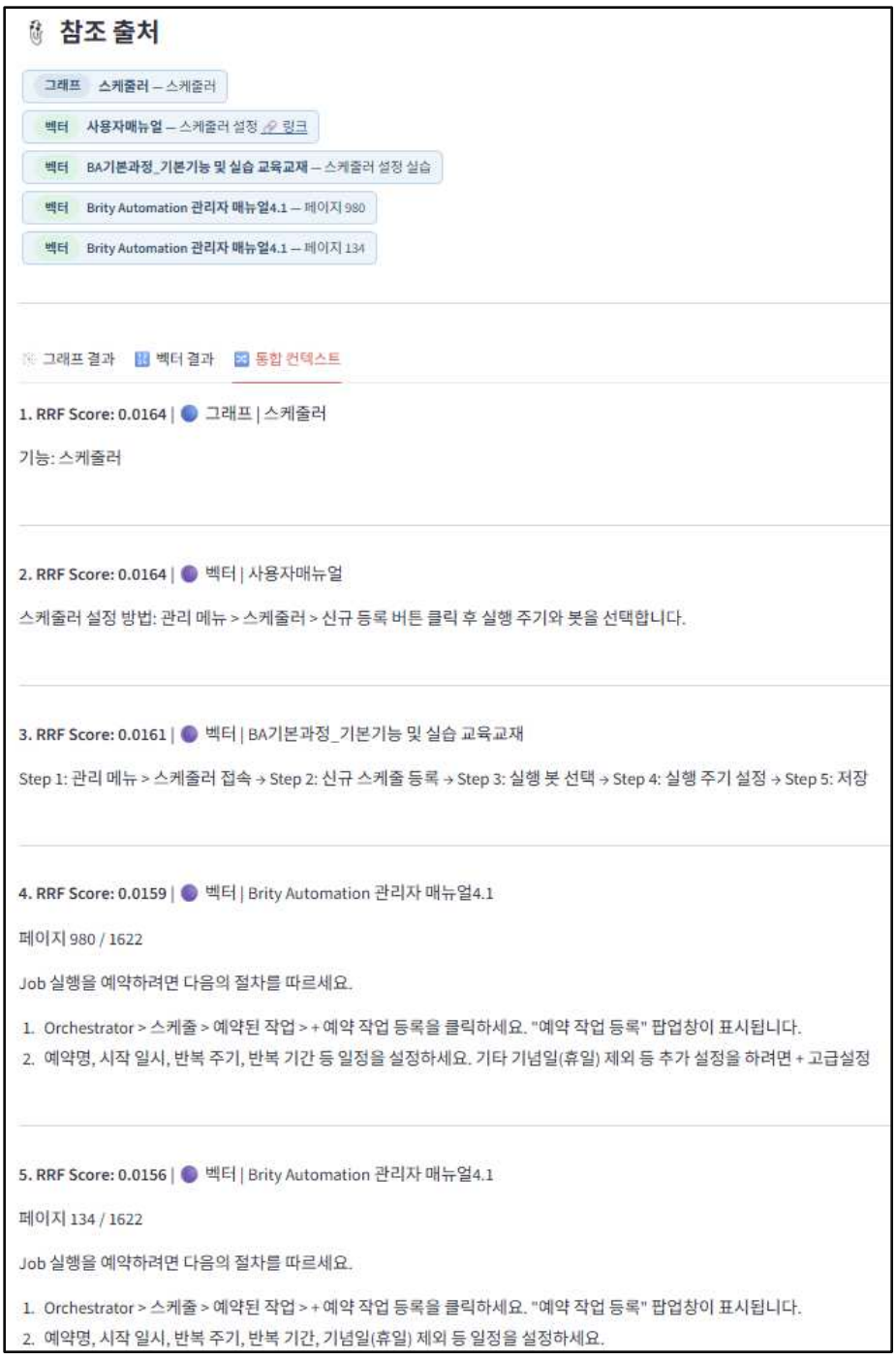

Figure 6: RRF Score Interface

4.5 Data Collection and Knowledge Indexing Interface

The data collection and indexing interface is shown in Figure 7. The HTML manual collection section provides URL links for each of the user manual, administration manual, installation manual, and troubleshooting guide, along with an 'Start HTML Manual Crawling' button. The PDF training materials processing section displays the existence status (file present/absent) for basic and advanced course PDF files, and initiates parsing and indexing via the 'Start PDF Training Materials Processing' button. Selecting the HTML manuals to be processed

and clicking the start crawling button initiates URL link crawling as shown in Figure 8.

Figure 7: Web Crawling and PDF Parsing Interface

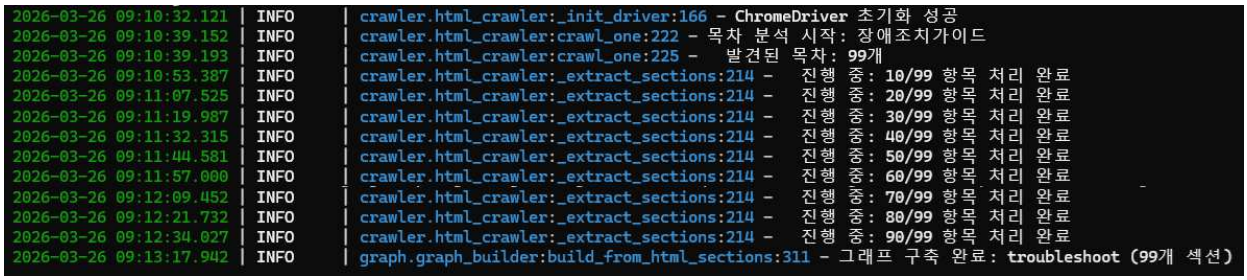

Figure 8: Crawling Progress Interface

PDF files can be uploaded to the designated folder, parsed, and their contents utilized for knowledge graph construction. The PDF parsing process not only extracts text but also preserves the structural information of documents to the greatest extent possible, thereby improving the accuracy of entity and relationship extraction. As shown in Figure 7, the list of uploaded PDF files can be confirmed, and selecting the PDF to be processed and clicking the processing button initiates parsing and storage in the database.

4.6 Knowledge Graph Visualization Interface

Entities and relationships extracted from collected and parsed documents are stored in Neo4j Graph Database to constitute the knowledge graph. The system provides a functionality to visually represent portions or the entirety of the constructed knowledge graph. The graph composition screen in Figure 9 displays the graph structure consisting of nodes and edges, and includes a function that shows the property information of a node when clicked. Node type statistics (Feature, ErrorCode, Resolution, Scenario, FAQ, Document, etc., with 6 total relationship types) are displayed numerically at the top. The interactive Plotly graph at the bottom visualizes Document nodes (user manual, administration manual, installation manual, troubleshooting guide), Feature nodes (scheduler, bot deployment, workflow, monitoring), ErrorCode nodes, Resolution nodes, and Scenario nodes—color-coded and displayed with relationship edges. From this interface, users can click on nodes to examine detailed properties and interactively explore relationship paths.

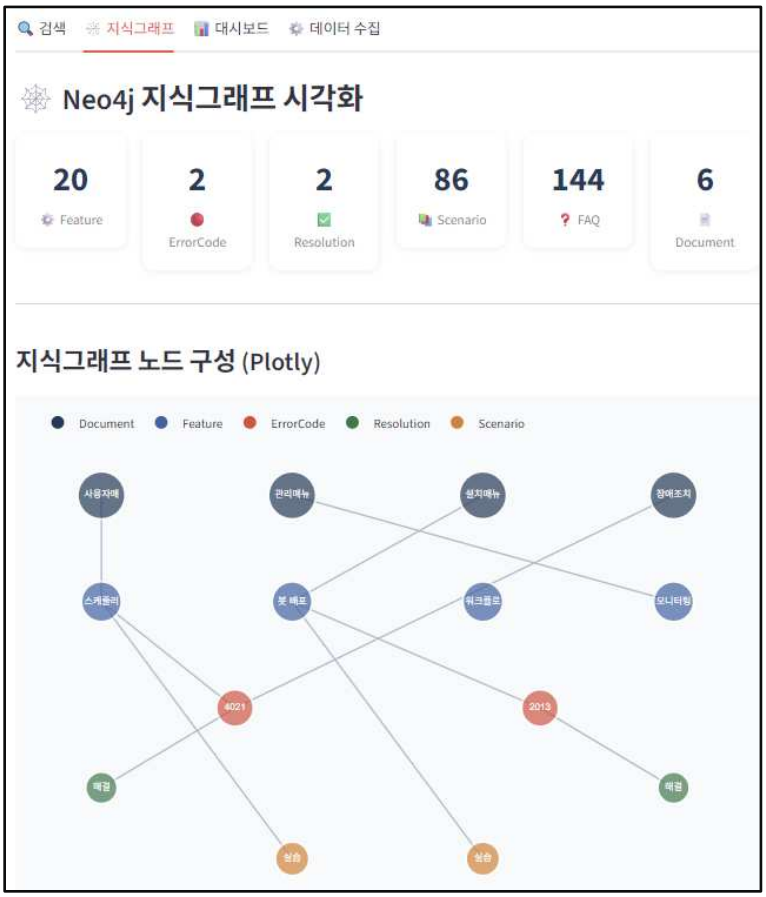

Figure 9: Knowledge Graph Composition Interface

4.7 Experimental

In this study, experiments were conducted using multiple evaluation metrics, including Precision@5 and Recall@5. Detailed results are currently undergoing further analysis.

5. Conclusion and Future Research

5.1 Research Conclusions

This study designed and implemented a Graph RAG system that constructs a knowledge graph from unstructured documents and leverages it to improve the performance of a Large Language Model (LLM)-based question answering system. To overcome the limitations of existing RAG systems—which rely on simple text retrieval and exhibit constraints in contextual understanding and deep reasoning—the proposed system represents entities and relationships within documents in the form of a structured knowledge graph, enabling the LLM to generate more accurate and reliable responses on the basis of richer background knowledge.

The implemented system comprises four modules: data collection and preprocessing, knowledge graph construction, Graph RAG module, and user interface module. In particular, diverse forms of unstructured documents were processed through web crawling and PDF parsing, and the extracted information was effectively managed through storage in Neo4j Graph Database. During the question answering process, information retrieved from the knowledge graph was integrated into LLM prompts, reducing LLM hallucination and improving response accuracy. Furthermore, by utilizing RRF scores to evaluate the importance of retrieval results and incorporating these evaluations into response generation, the reliability of information was enhanced.

The Graph RAG system developed through this research presented a novel approach for extracting core information from complex unstructured documents, structuring relationships among them, and maximizing the LLM's knowledge utilization capabilities. This is expected to expand the scope of LLM application in information retrieval and question answering, and to contribute to providing users with more intelligent and reliable information services.

5.2 Future Research Directions

Notwithstanding the contributions of this study, several future research directions are proposed for further improving the performance and applicability of Graph RAG systems.

First, automation and accuracy improvement of knowledge graph construction: While the current system relies on LLM assistance for entity and relationship extraction, the development of automated knowledge graph construction techniques capable of handling a wider variety of domains and more complex document structures is required. In particular, research aimed at improving the quality and scalability of knowledge graphs through automated ontology matching and inference rule generation is warranted.

Second, multi-modal data integration: Research extending the system to a question answering framework based on richer information by integrating not only text but also diverse modalities such as images, video, and audio into the knowledge graph is necessary. This could generate synergistic effects through integration with multi-modal LLMs.

Third, user feedback-based knowledge graph improvement: Research introducing a mechanism for collecting feedback generated during user question answering interactions and utilizing it for knowledge graph updating and refinement is needed. This would endow the system with self-improvement capabilities enabling continuous learning and development.

Fourth, real-time data processing and streaming RAG: To respond to the rapidly changing information environment, research on streaming RAG systems capable of processing real-time incoming data, reflecting it in the knowledge graph, and enabling immediate question answering is required. This would improve responsiveness to dynamic information sources such as news and social media.

Through these future research directions, the Graph RAG system is expected to evolve into an increasingly intelligent and powerful tool for information exploration and knowledge management.

References

Amugongo, L.M., Mascheroni, P., Brooks, S., Doering, S., & Seidel, J. (2025). Retrieval augmented generation for large language models in healthcare: *A systematic review. PLOS Digital Health*, 4, e0000877.

Bruch, S., Gai, S., & Ingber, A. (2023). An analysis of fusion functions for hybrid retrieval. *ACM Transactions on Information Systems*, 42(1), 1–35. https://doi.org/10.1145/3596512

Chen, J., Xiao, S., Zhang, P., Luo, K., Lian, D., & Liu, Z. (2024). BGE M3-Embedding: Multi-lingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation. *arXiv preprint arXiv:2402.03216*.

Cormack, G. V., Clarke, C. L. A., & Buettcher, S. (2009). Reciprocal rank fusion outperforms condorcet and individual rank learning methods. *Proceedings of the 32nd Annual International ACM SIGIR Conference (SIGIR '09)*, 758–759. https://doi.org/10.1145/1571941.1572114

Edge, D., Trinh, H., Cheng, N., Bradley, J., Chao, A., Mody, A., Truitt, S., & Larson, J. (2024). From local to global: A graph RAG approach to query-focused summarization. *arXiv preprint arXiv:2404.16130.* https://doi.org/10.48550/arXiv.2404.16130

Elkiran, H., & Rasheed, J. (2026). An empirical evaluation of retrieval, reranking, and similarity for a Q&A-based retrieval augmented generation system. *IEEE Access*, 14, 26053–26066. https://doi.org/10.1109/ACCESS.2026.3664852

Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., Dai, Y., Sun, J., & Wang, H. (2023). Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997.* https://doi.org/10.48550/arXiv.2312.10997

Gruber, T. R. (1993). A translation approach to portable ontology specifications. *Knowledge Acquisition*, 5(2), 199–220. https://doi.org/10.1006/knac.1993.1008

Hogan, A., Cochez, M., Rula, A., Darke, A., Maali, F., Gayo-Avello, D., & Sequeda, J. (2021). Knowledge Graphs. *ACM Computing Surveys*, 54(4), 1–37.

Jeong, C. (2023). Implementation of generative AI services utilizing LLM application architecture: Based on the RAG model and LangChain framework. *Journal of Intelligence and Information Systems*, 29(4), 129–164. https://dx.doi.org/10.13088/jiis.2023.29.4.129

Jeong, C. (2024). A study on implementation methods for graph agent-based Advanced RAG systems for knowledge-based QA improvement. *Knowledge Management Research*, 25(3), 99–119. https://doi.org/10.15813/kmr.2024.25.3.005

Jeong, C. (2026). Reliable AI and agentic RAG. *Communication Books*. ISBN 979-11-430-2335-3

Karpukhin, V., Oğuz, B., Min, S., Lewis, P., Wu, L., Edunov, S., Chen, D., & Yih, W. (2020). Dense passage retrieval for open-domain question answering. *Proceedings of EMNLP 2020*, 6769–6781. https://doi.org/10.18653/v1/2020.emnlp-main.550

Kim, T., & Lee, S. (2023). Design and evaluation of a RAG-based question answering system for legal documents. Journal of the Korea Information *Processing Society*, 30(4), 145–156. https://doi.org/10.3745/JKIICE.2023.30.4.145

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33, 9459–9474.

Ma, X., Liu, Z., Zhang, H., & Chen, Y. (2025). KA-RAG: Integrating knowledge graphs and agentic retrieval-augmented generation for an intelligent educational question-answering model. *Applied Sciences*, 15(23),

12547. https://doi.org/10.3390/app152312547

Noy, N. F., & McGuinness, D. L. (2001). Ontology development 101: A guide to creating your first ontology. *Stanford Knowledge Systems Laboratory Technical Report KSL-01-05*.

Ong, K. R., & Wong, W. P. (2025). Optimizing information retrieval in RAG through intelligent reranking and follow-up query predictions. *Research Square*. https://doi.org/10.21203/rs.3.rs-7506627/v1

Pan, S., Luo, L., Wang, Y., Chen, C., Wang, J., & Wu, X. (2024). Unifying large language models and knowledge graphs: A roadmap. *IEEE Transactions on Knowledge and Data Engineering*, 36(7), 3580–3599. https://doi.org/10.1109/TKDE.2024.3352100

Park, J., Choi, S., & Hwang, M. (2024). Evaluation of retrieval performance of the multilingual embedding model BGE-M3 on Korean technical documents. *Proceedings of the Korea Computer Congress 2024*, 51(1), 1234–1236.

Peng, B., Zhu, Y., Liu, Y., Bo, X., Shi, H., Hong, C., & Tang, J. (2024). Graph retrieval-augmented generation: A survey. *arXiv preprint arXiv:2408.08921*. https://doi.org/10.48550/arXiv.2408.08921

Petroni, F., Lewis, P., Piktus, A., Rocktäschel, T., Wu, Y., Miller, A. H., & Riedel, S. (2021). KILT: A benchmark for knowledge intensive language tasks. *Proceedings of NAACL 2021*, 2523–2544. https://doi.org/10.18653/v1/2021.naacl-main.200

Robinson, I., Webber, J., & Eifrem, E. (2015). Graph databases: New opportunities for connected data (2nd ed.). *O'Reilly Media*.

Singhal, A. (2012, May 16). Introducing the knowledge graph: Things, not strings. Google Blog. https://blog.google/products/search/introducing-knowledge-graph-things-not/

Vrandečić, D., & Krötzsch, M. (2014). Wikidata: A free collaborative knowledgebase. Communications of the ACM, 57(10), 78–85. https://doi.org/10.1145/2629489

Xu, Y., Chen, X., & Liu, Y. (2022). Knowledge graph-based technical manual retrieval for step-by-step task assistance. *Expert Systems with Applications*, 197, 116660. https://doi.org/10.1016/j.eswa.2022.116660

Zhang, N., Choubey, P. K., Fabbri, A., Bernadett-Shapiro, G., Zhang, R., Mitra, P., Xiong, C., & Wu, C.-S. (2024). SiReRAG: Indexing similar and related information for multihop reasoning. *arXiv preprint arXiv:2412.06206*. https://doi.org/10.48550/arXiv.2412.06206

Zhang, Q., Wang, H., Li, J., & Chen, Z. (2023). Knowledge graph-enhanced fault diagnosis for software documentation. *Information Processing & Managem*ent, 60(3), 103325. https://doi.org/10.1016/j.ipm.2023.103325